

Air Show Route Planning with Deconfliction Optimization - Final Report

Sean M. Kennedy (smkennedy5@wisc.edu)

Table of Contents

1. [Introduction](#)
2. [Mathematical Model](#)
3. [Solution](#)
4. [Results and Discussion](#)
5. [Conclusion](#)

Introduction

In this project, we introduce an optimization model for choreographing flight paths in air shows. Using a Mixed-Integer Programming (MIP) approach, we seek to create a solution that ensures aircraft separation while maximizing visual appeal based on audience preferences. The model incorporates various factors such as air show layout, aircraft performance data, planned stunts, and viewer interests.

Air shows have been a popular form of entertainment and aviation showcase for over a century, with the first air show held in 1909 [1]. As these events have grown in complexity and scale, the need for sophisticated planning tools has become increasingly apparent. The tragic crash at the Reno Air Races in 2011 highlighted the critical importance of safety in air show planning [2], spurring interest in more rigorous approaches to route planning and deconfliction.

The problem we are addressing is multifaceted, involving the intricate balance between safety requirements, aircraft capabilities, show choreography, and audience satisfaction. By optimizing these elements, we aim to enhance both the safety and spectator experience of air shows.

For this project, we are utilizing synthetically generated data to model an air show scenario. This approach allows us to test and refine our optimization model while mimicking the complexities of a real air show. The use of randomly generated values provides a robust dataset for model development and testing, circumventing the challenges of obtaining real-world air show planning data.

Our synthetic data structure includes:

1. A 10x10 grid representing the air show space
2. Three types of aircraft (Fighter, Acrobatic, Helicopter) with specific performance characteristics (max/min speed, turning radius, minimum separation)
3. Three types of maneuvers (Loop, Barrel Roll, Formation Flying) with associated duration, space requirements, and suitable aircraft types
4. 10 time slots to represent the show's duration
5. Audience preference data for different aircraft types at four viewing areas

To simplify the problem, we are making several assumptions:

1. Weather conditions are ideal and constant throughout the show.
2. All aircraft perform exactly as specified by their performance data.
3. The air show venue has no unexpected airspace restrictions beyond those defined by the grid.
4. Pilots can execute all planned maneuvers with perfect precision within the allocated time slots.
5. The discrete time slots and grid positions adequately represent continuous time and space for this problem.

References

- [1] Hallion, R. P. (2003). *Taking Flight: Inventing the Aerial Age from Antiquity through the First World War*. Oxford University Press.
- [2] National Transportation Safety Board. (2012). *Pilot/Race 177, The Galloping Ghost, North American P-51D, N79111, Reno, Nevada, September 16, 2011. Aircraft Accident Report NTSB/AAR-12/01*. Washington, DC.

2. Mathematical Model

We introduce a simpler and more complex model for our problem. First, the simpler model

2.1 Simpler Model

Modeling Assumptions

This model is designed to optimize the schedule and positioning of aircraft during an airshow. The key assumptions are:

1. The airshow takes place on a discrete 10x10 grid.
2. Time is divided into discrete slots (10 in total).
3. There are three types of aircraft: Fighter, Acrobatic, and Helicopter.
4. Each aircraft can perform three types of maneuvers: Loop, Barrel Roll, and Formation Flying.
5. Certain maneuvers are only suitable for specific aircraft types.
6. Audience preferences vary based on aircraft type and viewing area.
7. Aircraft have constraints on speed and separation.
8. In both models, slack variables help ensure constraint feasibility while being a penalized factor to maintain the solution's optimality. They provide flexibility where exact constraint satisfaction might be challenging.

Decision Variables

Let $x_{a,m,g,t}$ be a binary variable where:

- $a \in A$ (set of aircraft)
- $m \in M$ (set of maneuvers)
- $g \in G$ (set of grid positions)
- $t \in T$ (set of time slots)

$$x_{a,m,g,t} = \begin{cases} 1 & \text{if aircraft } a \text{ performs maneuver } m \text{ at grid position } g \text{ in time slot } t \\ 0 & \text{otherwise} \end{cases}$$

Let $y_{a,m,g,t}$ be a binary variable indicating if a maneuver continues beyond the current time slot (some maneuvers require multiple timeslots).

Let s_a be a variable representing the start time of aircraft a .

Let diff_{a_i,a_j} be a continuous variable representing the absolute difference in start times between aircraft a_i and a_j .

Constraints

1. Each aircraft performs exactly one maneuver at each time slot:

$$\sum_{m \in M} \sum_{g \in G} x_{a,m,g,t} = 1 \quad \forall a \in A, t \in T$$

2. Maneuver duration (with slack):

$$t + d_m - 1 \leq |T| + M(1 - y_{a,m,g,t}) + \text{slack}_{a,m,g,t} \quad \forall a \in A, m \in M, g \in G, t \in T$$

where d_m is the duration of maneuver m and M is a large constant.

3. Aircraft type suitability for maneuvers:

$$\sum_{m \notin S_a} x_{a,m,g,t} = 0 \quad \forall a \in A, g \in G, t \in T$$

where S_a is the set of suitable maneuvers for aircraft a .

4. No two aircraft can be in the same position at the same time:

$$\sum_{m \in M} x_{a_i,m,g,t} + \sum_{m \in M} x_{a_j,m,g,t} \leq 1 \quad \forall a_i, a_j \in A (i \neq j), g \in G, t \in T$$

5. Start time constraints:

$$s_a \leq t \cdot x_{a,m,g,t} + M(1 - x_{a,m,g,t}) \quad \forall a \in A, m \in M, g \in G, t \in T$$

6. Linearization of absolute value for start time differences:

- $\text{diff}_{a_i,a_j} \geq s_{a_i} - s_{a_j} \quad \forall a_i, a_j \in A (i \neq j)$
- $\text{diff}_{a_i,a_j} \geq s_{a_j} - s_{a_i} \quad \forall a_i, a_j \in A (i \neq j)$

Objective Function

The objective is to maximize audience preference while minimizing start time differences and slack:

$$\text{maximize } \sum_{a \in A} \sum_{m \in M} \sum_{g \in V} \sum_{t \in T} p_{g,a} \cdot x_{a,m,g,t} - 0.1 \sum_{a_i, a_j \in A, i \neq j} \text{diff}_{a_i,a_j} - 100 \sum_{a,m,g,t} \text{slack}_{a,m,g,t}$$

where $p_{g,a}$ is the audience preference for aircraft type a at viewing area g , and V is the set of viewing areas.

The diff_{a_i,a_j} term represents the absolute difference in start times between each pair of aircraft, encouraging aircraft to start at similar times.

Standard Form

The complete problem is:

$$\begin{aligned} & \text{maximize } \sum_{a \in A} \sum_{m \in M} \sum_{g \in V} \sum_{t \in T} p_{g,a} \cdot x_{a,m,g,t} - 0.1 \sum_{a_i, a_j \in A, i \neq j} \text{diff}_{a_i,a_j} - 100 \sum_{a,m,g,t} \text{slack}_{a,m,g,t} \\ & \text{subject to: } \sum_{m \in M} \sum_{g \in G} x_{a,m,g,t} = 1 && \forall a \in A \\ & t + d_m - 1 \leq |T| + M(1 - y_{a,m,g,t}) + \text{slack}_{a,m,g,t} && \forall a \in A \\ & x_{a,m,g,t} \leq y_{a,m,g,t} && \forall a \in A \\ & \sum_{m \notin S_a} x_{a,m,g,t} = 0 && \forall a \in A \\ & \sum_{m \in M} x_{a_i,m,g,t} + \sum_{m \in M} x_{a_j,m,g,t} \leq 1 && \forall a_i, a_j \\ & s_a \leq t \cdot x_{a,m,g,t} + M(1 - x_{a,m,g,t}) && \forall a \in A \\ & \text{diff}_{a_i,a_j} \geq s_{a_i} - s_{a_j} && \forall a_i, a_j \\ & \text{diff}_{a_i,a_j} \geq s_{a_j} - s_{a_i} && \forall a_i, a_j \\ & x_{a,m,g,t}, y_{a,m,g,t} \in \{0, 1\} && \forall a \in A \\ & s_a, \text{slack}_{a,m,g,t}, \text{diff}_{a_i,a_j} \geq 0 && \forall a, a_i, a_j \end{aligned}$$

Model Type

This is a Mixed Integer Linear Programming (MILP) problem. It contains both binary variables ($x_{a,m,g,t}$ and $y_{a,m,g,t}$) and continuous variables (s_a , $\text{slack}_{a,m,g,t}$, and diff_{a_1,a_2}). The objective function and all constraints are linear, making it suitable for solving with MILP solvers like Gurobi.

2.2 Complex Model

This section outlines the changes and additions made to the previous model to enhance the realism of the airshow optimization problem.

Updated Modeling Assumptions

The core assumptions remain the same, with the following additions:

1. Aircraft movement is now explicitly modeled and encouraged.
2. The model now considers the physical positioning of aircraft on the grid.
3. Maneuver changes between time slots are tracked and incentivized.

New Decision Variables

In addition to the previously defined variables, we introduce:

- Continuous variables representing the x and y coordinates of aircraft a at time t :
 - $\text{pos}_{x,a,t}, \text{pos}_{y,a,t} \in [1, 10]$
- Continuous variables for linearizing the absolute value in speed constraints:
 - $dx_{a,t}^+, dx_{a,t}^-, dy_{a,t}^+, dy_{a,t}^- \geq 0$
- Binary variables for minimum separation constraints:
 - $\text{sep}_{x,a_i,a_j,t}, \text{sep}_{y,a_i,a_j,t} \in \{0, 1\}$
- Binary variable indicating if aircraft a changes maneuvers between time t and $t + 1$:
 - $\text{change}_{a,t} \in \{0, 1\}$
- Continuous variables for linearizing maneuver changes:
 - $\text{diff}_{a,m,g,t}^+, \text{diff}_{a,m,g,t}^- \geq 0$

New and Modified Constraints

1. Position constraints:

- $x_{a,m,g,t} \Rightarrow \text{pos}_{x,a,t} = g_x \quad \forall a \in A, m \in M, g \in G, t \in T$
- $x_{a,m,g,t} \Rightarrow \text{pos}_{y,a,t} = g_y \quad \forall a \in A, m \in M, g \in G, t \in T$

2. Speed constraints:

- $\text{pos}_{x,a,t+1} - \text{pos}_{x,a,t} = dx_{a,t}^+ - dx_{a,t}^- \quad \forall a \in A, t \in T \setminus \{|T|\}$
- $\text{pos}_{y,a,t+1} - \text{pos}_{y,a,t} = dy_{a,t}^+ - dy_{a,t}^- \quad \forall a \in A, t \in T \setminus \{|T|\}$
- $dx_{a,t}^+ + dx_{a,t}^- + dy_{a,t}^+ + dy_{a,t}^- \leq v_a^{\max} \quad \forall a \in A, t \in T \setminus \{|T|\}$
- $dx_{a,t}^+ + dx_{a,t}^- + dy_{a,t}^+ + dy_{a,t}^- \geq v_a^{\min} \quad \forall a \in A, t \in T \setminus \{|T|\}$

3. Minimum separation constraints:

- $\text{pos}_{x,a_i,t} - \text{pos}_{x,a_j,t} + M \cdot \text{sep}_{x,a_i,a_j,t} \geq \max(d_{a_i}^{\min}, d_{a_j}^{\min}) \quad \forall a_i, a_j \in A (i \neq j), t \in T$
- $\text{pos}_{x,a_j,t} - \text{pos}_{x,a_i,t} + M \cdot (1 - \text{sep}_{x,a_i,a_j,t}) \geq \max(d_{a_i}^{\min}, d_{a_j}^{\min}) \quad \forall a_i, a_j \in A (i \neq j), t \in T$
- $\text{pos}_{y,a_i,t} - \text{pos}_{y,a_j,t} + M \cdot \text{sep}_{y,a_i,a_j,t} \geq \max(d_{a_i}^{\min}, d_{a_j}^{\min}) \quad \forall a_i, a_j \in A (i \neq j), t \in T$

- $pos_{y,a_j,t} - pos_{y,a_i,t} + M \cdot (1 - sep_{y,a_i,a_j,t}) \geq \max(d_{a_i}^{min}, d_{a_j}^{min}) \quad \forall a_i, a_j \in A (i \neq j), t \in T$
- $sep_{x,a_i,a_j,t} + sep_{y,a_i,a_j,t} \leq 1 \quad \forall a_i, a_j \in A (i \neq j), t \in T$

4. Maneuver change constraints:

- $x_{a,m,g,t+1} - x_{a,m,g,t} = diff_{a,m,g,t}^+ - diff_{a,m,g,t}^- \quad \forall a \in A, m \in M, g \in G, t \in T \setminus \{|T|\}$
- $\sum_{m \in M} \sum_{g \in G} (diff_{a,m,g,t}^+ + diff_{a,m,g,t}^-) \leq 2 \cdot change_{a,t} \quad \forall a \in A, t \in T \setminus \{|T|\}$

Updated Objective Function

The objective function now includes terms to encourage movement and maneuver changes:

$$\begin{aligned}
\text{maximize} \quad & \sum_{a \in A} \sum_{m \in M} \sum_{g \in V} \sum_{t \in T} p_{g,a} \cdot x_{a,m,g,t}^+ \\
& 0.1 \sum_{a \in A} \sum_{t \in T \setminus \{|T|\}} (dx_{a,t}^+ + dx_{a,t}^- + dy_{a,t}^+ + dy_{a,t}^-) + \\
& 0.5 \sum_{a \in A} \sum_{t \in T \setminus \{|T|\}} change_{a,t} - \\
& 0.1 \sum_{a_i, a_j \in A, i \neq j} diff_{a_i, a_j} - \\
& 100 \sum_{a \in A} \sum_{m \in M} \sum_{g \in G} \sum_{t \in T} slack_{a,m,g,t}
\end{aligned}$$

The new terms incentivize aircraft movement and maneuver changes, while still considering audience preferences, start time differences, and minimizing slack usage. In both the simple and complex model, the cost coefficients are parameters that can be changed based on desired characteristics.

Model Complexity

While the core structure remains a MILP problem, the updated model has more complexity, which may lead to longer solve times but should result in a more realistic and dynamic airshow plan.

3. Solution

First we implement the simpler model.

```
In [ ]: using Random, StatsBase, JuMP, Gurobi
```

```
Random.seed!(123) # Set a seed for reproducibility

# Define grid size for the 10x10 airshow area
grid_size = (10, 10)

# Generate aircraft data
# Three types of aircraft: Fighter, Acrobatic, Helicopter
aircraft_types = ["Fighter", "Acrobatic", "Helicopter"]
aircraft_instances = [
    (type=t, id=i) for t in aircraft_types for i in 1:1
]
# Define characteristics for each aircraft type
aircraft_data = Dict{
    type => (
        max_speed = rand(400:600),
        min_speed = rand(100:300),
        turning_radius = rand(50:150),
        min_separation = rand(50:100)
    ) for type in aircraft_types
}

# Generate maneuver data
# Three types of maneuvers: Loop, Barrel Roll, Formation Flying
maneuvers = ["Loop", "Barrel Roll", "Formation Flying"]
maneuver_data = Dict{
    maneuver => (
        duration = rand(1:3),
        required_space = (1, 1),
        suitable_aircraft = if maneuver == "Loop"
            ["Fighter", "Acrobatic"]
        elseif maneuver == "Barrel Roll"
            ["Fighter", "Acrobatic", "Helicopter"]
        else # Formation Flying
            ["Fighter", "Acrobatic", "Helicopter"]
        end
    ) for maneuver in maneuvers
}

# Generate time slots (10 in total, as per the updated model)
num_time_slots = 10

# Generate audience preference data for each viewing area and aircraft type
viewing_areas = [(1,1), (1,10), (10,1), (10,10)]
audience_preferences = Dict{
    (area, aircraft_type) => rand()
    for area in viewing_areas, aircraft_type in aircraft_types
}

# Create the optimization model using Gurobi solver
model = Model(Gurobi.Optimizer)

# Set Gurobi parameters
set_optimizer_attribute(model, "TimeLimit", 300) # 5 minute time limit
```

```

set_optimizer_attribute(model, "MIPGap", 0.05) # Accept solutions within 5% of optimal

# Define sets for the model
aircraft = aircraft_instances
time_slots = 1:num_time_slots
grid = [(i, j) for i in 1:grid_size[1], j in 1:grid_size[2]]

# Constants
M = num_time_slots # Big M constant for logical constraints

# Decision Variables
# x[a,m,g,t]: 1 if aircraft a performs maneuver m at grid position g in time slot t, 0 otherwise
@variable(model, x[a in aircraft, m in maneuvers, g in grid, t in time_slots], Bin)
# start_time[a]: The start time of aircraft a
@variable(model, start_time[a in aircraft] >= 0)
# y[a,m,g,t]: 1 if maneuver m by aircraft a at grid position g continues beyond time slot t, 0 otherwise
@variable(model, y[a in aircraft, m in maneuvers, g in grid, t in time_slots], Bin)

# Variables for linearizing the absolute value in the objective
@variable(model, time_diff[a1 in aircraft, a2 in aircraft] >= 0)

# Slack variables for relaxing constraints
@variable(model, slack_duration[a in aircraft, m in maneuvers, g in grid, t in time_slots] >= 0)

# Constraints

# Each aircraft performs exactly one maneuver at each time slot
@constraint(model, [a in aircraft, t in time_slots],
    sum(x[a, m, g, t] for m in maneuvers, g in grid) == 1)

# Maneuver duration constraint (relaxed with slack)
@constraint(model, [a in aircraft, m in maneuvers, g in grid, t in time_slots],
    t + maneuver_data[m].duration - 1 <= num_time_slots + M * (1 - y[a, m, g, t]) + slack_duration[a, m, g, t])
@constraint(model, [a in aircraft, m in maneuvers, g in grid, t in time_slots],
    x[a, m, g, t] <= y[a, m, g, t])

# Aircraft type suitability for maneuvers
@constraint(model, [a in aircraft, m in maneuvers, g in grid, t in time_slots],
    sum(x[a, m, g, t] for m in maneuvers if !(a.type in maneuver_data[m].suitable_aircraft)) == 0)

# No two aircraft can be in the same position at the same time
@constraint(model, [a1 in aircraft, a2 in aircraft, g in grid, t in time_slots; a1 != a2],
    sum(x[a1, m1, g, t] for m1 in maneuvers) + sum(x[a2, m2, g, t] for m2 in maneuvers) <= 1)

```



```

# Start time constraints
@constraint(model, [a in aircraft, m in maneuvers, g in grid, t in time_slots],
    start_time[a] <= t * x[a, m, g, t] + M * (1 - x[a, m, g, t]))

# Linearization of absolute value for start time differences
@constraint(model, [a1 in aircraft, a2 in aircraft; a1 != a2],
    time_diff[a1, a2] >= start_time[a1] - start_time[a2])
@constraint(model, [a1 in aircraft, a2 in aircraft; a1 != a2],
    time_diff[a1, a2] >= start_time[a2] - start_time[a1])

# Objective: Maximize audience preference, minimize start time differences, and minimize slack
@objective(model, Max,
    sum(audience_preferences[(area, a.type)] * x[a, m, area, t]
        for a in aircraft, m in maneuvers, area in viewing_areas, t in time_slots) -
    0.1 * sum(time_diff[a1, a2] for a1 in aircraft, a2 in aircraft if a1 != a2) -
    100 * sum(slack_duration)
)

# Solve the model
optimize!(model)

# Print results
if termination_status(model) == MOI.OPTIMAL || termination_status(model) == MOI.TIME_LIMIT
    println("Solution found")
    println("Termination status: ", termination_status(model))
    println("Solve time: ", solve_time(model))
    println("Objective value: ", objective_value(model))

    # Print schedule for each aircraft
    for a in aircraft, m in maneuvers, g in grid, t in time_slots
        if value(x[a, m, g, t]) > 0.5
            println("Aircraft $(a.id) ($(a.type)) performs $(m) at position $(g) at time $(t)")
        end
    end

    # Print start times for each aircraft
    for a in aircraft
        println("Aircraft $(a.id) ($(a.type)) starts at time $(value(start_time[a]))")
    end

    # Print total slack variable usage
    total_duration_slack = sum(value(slack_duration[a,m,g,t]) for a in aircraft, m in maneuvers, g in grid, t in time_slots)
    println("Total duration slack used: $total_duration_slack")
else
    println("No solution found")
    println("Termination status: ", termination_status(model))
end

```


Set parameter Username
 Academic license - for non-commercial use only - expires 2025-03-07
 Set parameter TimeLimit to value 300
 Set parameter MIPGap to value 0.05
 Set parameter MIPGap to value 0.05
 Set parameter TimeLimit to value 300
 Gurobi Optimizer version 11.0.2 build v11.0.2rc0 (win64 - Windows 10.0 (1904 5.2))

CPU model: Intel(R) Core(TM) i7-10875H CPU @ 2.30GHz, instruction set [SSE2|AVX|AVX2]
 Thread count: 8 physical cores, 16 logical processors, using up to 16 threads

Optimize a model with 42042 rows, 27012 columns and 101136 nonzeros
 Model fingerprint: 0x888af74d
 Variable types: 9012 continuous, 18000 integer (18000 binary)
 Coefficient statistics:
 Matrix range [1e+00, 1e+01]
 Objective range [9e-02, 1e+02]
 Bounds range [0e+00, 0e+00]
 RHS range [1e+00, 2e+01]
 Found heuristic solution: objective -300.0000000
 Presolve removed 41939 rows and 26712 columns
 Presolve time: 0.21s
 Presolved: 103 rows, 300 columns, 600 nonzeros
 Found heuristic solution: objective -79.9484886
 Variable types: 0 continuous, 300 integer (300 binary)

Explored 0 nodes (0 simplex iterations) in 0.23 seconds (0.14 work units)
 Thread count was 16 (of 16 available processors)

Solution count 2: -79.9485 -300

Optimal solution found (tolerance 5.00e-02)
 Best objective -7.994848858806e+01, best bound -7.640842490103e+01, gap 4.4279%

User-callback calls 480, time in user-callback 0.01 sec
 Solution found
 Termination status: OPTIMAL

Academic license - for non-commercial use only - expires 2025-03-07
 Set parameter TimeLimit to value 300
 Set parameter MIPGap to value 0.05
 Set parameter MIPGap to value 0.05
 Set parameter TimeLimit to value 300
 Gurobi Optimizer version 11.0.2 build v11.0.2rc0 (win64 - Windows 10.0 (1904 5.2))

CPU model: Intel(R) Core(TM) i7-10875H CPU @ 2.30GHz, instruction set [SSE2|AVX|AVX2]
 Thread count: 8 physical cores, 16 logical processors, using up to 16 threads

Optimize a model with 42042 rows, 27012 columns and 101136 nonzeros
 Model fingerprint: 0x888af74d

Variable types: 9012 continuous, 18000 integer (18000 binary)

Coefficient statistics:

Matrix range [1e+00, 1e+01]

Objective range [9e-02, 1e+02]

Bounds range [0e+00, 0e+00]

RHS range [1e+00, 2e+01]

Found heuristic solution: objective -300.0000000

Presolve removed 41939 rows and 26712 columns

Presolve time: 0.21s

Presolved: 103 rows, 300 columns, 600 nonzeros

Found heuristic solution: objective -79.9484886

Variable types: 0 continuous, 300 integer (300 binary)

Explored 0 nodes (0 simplex iterations) in 0.23 seconds (0.14 work units)

Thread count was 16 (of 16 available processors)

Solution count 2: -79.9485 -300

Optimal solution found (tolerance 5.00e-02)

Best objective -7.994848858806e+01, best bound -7.640842490103e+01, gap 4.4279%

User-callback calls 480, time in user-callback 0.01 sec

Solution found

Termination status: OPTIMAL

Solve time: 0.2330000400543213

Objective value: -79.94848858806165

Aircraft 1 (Fighter) performs Loop at position (1, 10) at time 8

Aircraft 1 (Fighter) performs Loop at position (10, 10) at time 2

Aircraft 1 (Fighter) performs Loop at position (10, 10) at time 3

Aircraft 1 (Fighter) performs Loop at position (10, 10) at time 10

Aircraft 1 (Fighter) performs Barrel Roll at position (10, 10) at time 6

Aircraft 1 (Fighter) performs Barrel Roll at position (10, 10) at time 7

Aircraft 1 (Fighter) performs Formation Flying at position (10, 10) at time 1

Aircraft 1 (Fighter) performs Formation Flying at position (10, 10) at time 4

Aircraft 1 (Fighter) performs Formation Flying at position (10, 10) at time 5

Aircraft 1 (Fighter) performs Formation Flying at position (10, 10) at time 9

Aircraft 1 (Acrobatic) performs Loop at position (10, 1) at time 1

Aircraft 1 (Acrobatic) performs Loop at position (10, 1) at time 2

Aircraft 1 (Acrobatic) performs Loop at position (10, 1) at time 10

Aircraft 1 (Acrobatic) performs Barrel Roll at position (10, 1) at time 3

Aircraft 1 (Acrobatic) performs Barrel Roll at position (10, 1) at time 6

Aircraft 1 (Acrobatic) performs Barrel Roll at position (10, 1) at time 8

Aircraft 1 (Acrobatic) performs Formation Flying at position (10, 1) at time 4

Aircraft 1 (Acrobatic) performs Formation Flying at position (10, 1) at time 5

Aircraft 1 (Acrobatic) performs Formation Flying at position (10, 1) at time 7

Aircraft 1 (Acrobatic) performs Formation Flying at position (10, 1) at time 9

Aircraft 1 (Helicopter) performs Barrel Roll at position (1, 10) at time 1

Aircraft 1 (Helicopter) performs Barrel Roll at position (1, 10) at time 2

Aircraft 1 (Helicopter) performs Barrel Roll at position (1, 10) at time 3

Aircraft 1 (Helicopter) performs Barrel Roll at position (1, 10) at time 5

Aircraft 1 (Helicopter) performs Barrel Roll at position (1, 10) at time 6

Aircraft 1 (Helicopter) performs Barrel Roll at position (1, 10) at time 10
Aircraft 1 (Helicopter) performs Barrel Roll at position (10, 10) at time 8
Aircraft 1 (Helicopter) performs Formation Flying at position (1, 10) at time 4
Aircraft 1 (Helicopter) performs Formation Flying at position (1, 10) at time 7
Aircraft 1 (Helicopter) performs Formation Flying at position (1, 10) at time 9
Aircraft 1 (Fighter) starts at time 0.0
Aircraft 1 (Acrobatic) starts at time 0.0
Aircraft 1 (Helicopter) starts at time 0.0
Total duration slack used: 1.0

Next, we implement the more complex model.

```
In [ ]: using Random, StatsBase, JuMP, Gurobi
```

```
Random.seed!(123) # Set a seed for reproducibility

# Define grid size for the 10x10 airshow area
grid_size = (10, 10)

# Generate aircraft data
aircraft_types = ["Fighter", "Acrobatic", "Helicopter"]
aircraft_instances = [
    (type=t, id=i) for t in aircraft_types for i in 1:1
]
# Define characteristics for each aircraft type
aircraft_data = Dict{
    type => (
        max_speed = rand(2:3), # Maximum speed (grid cells per time slot)
        min_speed = 1, # Minimum speed (grid cells per time slot)
        turning_radius = 1, # Not used in the current model
        min_separation = 1 # Minimum separation distance between aircraft
    ) for type in aircraft_types
}

# Generate maneuver data
maneuvers = ["Loop", "Barrel Roll", "Formation Flying"]
maneuver_data = Dict{
    maneuver => (
        duration = rand(1:2), # Duration of the maneuver (in time slots)
        required_space = (1, 1), # Not used in the current model
        suitable_aircraft = if maneuver == "Loop"
            ["Fighter", "Acrobatic"]
        elseif maneuver == "Barrel Roll"
            ["Fighter", "Acrobatic", "Helicopter"]
        else # Formation Flying
            ["Fighter", "Acrobatic", "Helicopter"]
        end
    ) for maneuver in maneuvers
}

# Generate time slots (10 in total)
num_time_slots = 10

# Generate audience preference data for each viewing area and aircraft type
viewing_areas = [(1,1), (1,10), (10,1), (10,10)]
audience_preferences = Dict{
    (area, aircraft_type) => rand()
    for area in viewing_areas, aircraft_type in aircraft_types
}

# Create the optimization model using Gurobi solver
model = Model(Gurobi.Optimizer)

# Set Gurobi parameters
set_optimizer_attribute(model, "TimeLimit", 300) # 5 minute time limit
set_optimizer_attribute(model, "MIPGap", 0.05) # Accept solutions within 5% of optimal
```

```

# Define sets for the model
aircraft = aircraft_instances
time_slots = 1:num_time_slots
grid = [(i, j) for i in 1:grid_size[1], j in 1:grid_size[2]]

# Constants
M = maximum(grid_size) # Big M constant, just needs to be as big as the grid
size

# Decision Variables
# x[a,m,g,t]: 1 if aircraft a performs maneuver m at grid position g in time s
# lot t, 0 otherwise
@variable(model, x[a in aircraft, m in maneuvers, g in grid, t in time_slots],
Bin)
# start_time[a]: The start time of aircraft a
@variable(model, start_time[a in aircraft] >= 0)
# y[a,m,g,t]: 1 if maneuver m by aircraft a at grid position g continues beyon
d time slot t, 0 otherwise
@variable(model, y[a in aircraft, m in maneuvers, g in grid, t in time_slots],
Bin)

# Variables for aircraft position
@variable(model, 1 <= pos_x[a in aircraft, t in time_slots] <= grid_size[1])
@variable(model, 1 <= pos_y[a in aircraft, t in time_slots] <= grid_size[2])

# Variables for linearizing absolute value in speed constraints
@variable(model, dx_plus[a in aircraft, t in 1:num_time_slots-1] >= 0)
@variable(model, dx_minus[a in aircraft, t in 1:num_time_slots-1] >= 0)
@variable(model, dy_plus[a in aircraft, t in 1:num_time_slots-1] >= 0)
@variable(model, dy_minus[a in aircraft, t in 1:num_time_slots-1] >= 0)

# Variables for minimum separation
@variable(model, sep_x[a1 in aircraft, a2 in aircraft, t in time_slots], Bin)
@variable(model, sep_y[a1 in aircraft, a2 in aircraft, t in time_slots], Bin)

# Variables for linearizing the absolute value in the objective
@variable(model, time_diff[a1 in aircraft, a2 in aircraft] >= 0)

# Slack variables for relaxing constraints
@variable(model, slack_duration[a in aircraft, m in maneuvers, g in grid, t in
time_slots] >= 0)

# Variables for maneuver changes
@variable(model, maneuver_change[a in aircraft, t in 1:num_time_slots-1], Bin)
@variable(model, maneuver_diff_plus[a in aircraft, m in maneuvers, g in grid,
t in 1:num_time_slots-1] >= 0)
@variable(model, maneuver_diff_minus[a in aircraft, m in maneuvers, g in grid,
t in 1:num_time_slots-1] >= 0)

# Constraints

# Each aircraft performs exactly one maneuver at each time slot
@constraint(model, [a in aircraft, t in time_slots],
sum(x[a, m, g, t] for m in maneuvers, g in grid) == 1)

```

```

# Maneuver duration (relaxed with slack)
@constraint(model, [a in aircraft, m in maneuvers, g in grid, t in time_slots],
    t + maneuver_data[m].duration - 1 <= num_time_slots + M * (1 - y[a, m, g, t]) + slack_duration[a, m, g, t])
@constraint(model, [a in aircraft, m in maneuvers, g in grid, t in time_slots],
    x[a, m, g, t] <= y[a, m, g, t])

# Aircraft type suitability for maneuvers
@constraint(model, [a in aircraft, m in maneuvers, g in grid, t in time_slots],
    sum(x[a, m, g, t] for m in maneuvers if !(a.type in maneuver_data[m].suitable_aircraft)) == 0)

# No two aircraft can be in the same position at the same time
@constraint(model, [a1 in aircraft, a2 in aircraft, g in grid, t in time_slots; a1 != a2],
    sum(x[a1, m1, g, t] for m1 in maneuvers) + sum(x[a2, m2, g, t] for m2 in maneuvers) <= 1)

# Start time constraints
@constraint(model, [a in aircraft, m in maneuvers, g in grid, t in time_slots],
    start_time[a] <= t * x[a, m, g, t] + M * (1 - x[a, m, g, t]))

# Linearization of absolute value for start time differences
@constraint(model, [a1 in aircraft, a2 in aircraft; a1 != a2],
    time_diff[a1, a2] >= start_time[a1] - start_time[a2])
@constraint(model, [a1 in aircraft, a2 in aircraft; a1 != a2],
    time_diff[a1, a2] >= start_time[a2] - start_time[a1])

# Constraints for aircraft position
@constraint(model, [a in aircraft, m in maneuvers, t in time_slots, g in grid],
    x[a, m, g, t] => {pos_x[a, t] == g[1]})
@constraint(model, [a in aircraft, m in maneuvers, t in time_slots, g in grid],
    x[a, m, g, t] => {pos_y[a, t] == g[2]})

# Speed constraints (linearized)
@constraint(model, [a in aircraft, t in 1:num_time_slots-1],
    pos_x[a, t+1] - pos_x[a, t] == dx_plus[a, t] - dx_minus[a, t])
@constraint(model, [a in aircraft, t in 1:num_time_slots-1],
    pos_y[a, t+1] - pos_y[a, t] == dy_plus[a, t] - dy_minus[a, t])

@constraint(model, [a in aircraft, t in 1:num_time_slots-1],
    dx_plus[a, t] + dx_minus[a, t] + dy_plus[a, t] + dy_minus[a, t] <= aircraft_data[a.type].max_speed)
@constraint(model, [a in aircraft, t in 1:num_time_slots-1],
    dx_plus[a, t] + dx_minus[a, t] + dy_plus[a, t] + dy_minus[a, t] >= aircraft_data[a.type].min_speed)

# Minimum separation constraints (linearized)
@constraint(model, [a1 in aircraft, a2 in aircraft, t in time_slots; a1 != a2],

```



```

        pos_x[a1, t] - pos_x[a2, t] + M * sep_x[a1, a2, t] >= max(aircraft_data[a1.type].min_separation, aircraft_data[a2.type].min_separation))
    @constraint(model, [a1 in aircraft, a2 in aircraft, t in time_slots; a1 != a2],
        pos_x[a2, t] - pos_x[a1, t] + M * (1 - sep_x[a1, a2, t]) >= max(aircraft_data[a1.type].min_separation, aircraft_data[a2.type].min_separation))
    @constraint(model, [a1 in aircraft, a2 in aircraft, t in time_slots; a1 != a2],
        pos_y[a1, t] - pos_y[a2, t] + M * sep_y[a1, a2, t] >= max(aircraft_data[a1.type].min_separation, aircraft_data[a2.type].min_separation))
    @constraint(model, [a1 in aircraft, a2 in aircraft, t in time_slots; a1 != a2],
        pos_y[a2, t] - pos_y[a1, t] + M * (1 - sep_y[a1, a2, t]) >= max(aircraft_data[a1.type].min_separation, aircraft_data[a2.type].min_separation))
    @constraint(model, [a1 in aircraft, a2 in aircraft, t in time_slots; a1 != a2],
        sep_x[a1, a2, t] + sep_y[a1, a2, t] <= 1)

# Linearized constraints for maneuver changes
@constraint(model, [a in aircraft, m in maneuvers, g in grid, t in 1:num_time_slots-1],
    x[a, m, g, t+1] - x[a, m, g, t] == maneuver_diff_plus[a, m, g, t] - maneuver_diff_minus[a, m, g, t])

@constraint(model, [a in aircraft, t in 1:num_time_slots-1],
    sum(maneuver_diff_plus[a, m, g, t] + maneuver_diff_minus[a, m, g, t] for m in maneuvers, g in grid) <= 2 * maneuver_change[a, t])

# Objective: Maximize audience preference, minimize start time differences, encourage movement and maneuver changes
@objective(model, Max,
    sum(audience_preferences[(area, a.type)] * x[a, m, area, t]
        for a in aircraft, m in maneuvers, area in viewing_areas, t in time_slots) +
    0.1 * sum(dx_plus[a, t] + dx_minus[a, t] + dy_plus[a, t] + dy_minus[a, t]
        for a in aircraft, t in 1:num_time_slots-1) +
    0.5 * sum(maneuver_change[a, t] for a in aircraft, t in 1:num_time_slots-1) -
    0.1 * sum(time_diff[a1, a2] for a1 in aircraft, a2 in aircraft if a1 != a2) -
    100 * sum(slack_duration)
)

# Solve the model
optimize!(model)

# Print results
if termination_status(model) == MOI.OPTIMAL || termination_status(model) == MOI.TIME_LIMIT
    println("Solution found")
    println("Termination status: ", termination_status(model))
    println("Solve time: ", solve_time(model))
    println("Objective value: ", objective_value(model))

    println("\nSchedule of Maneuvers:")
    for a in aircraft, t in time_slots

```

```

        for m in maneuvers, g in grid
            if value(x[a, m, g, t]) > 0.5
                println("Time $t: Aircraft $(a.id) ($(a.type)) performs $m at
position $g")
            end
        end
    end

    println("\nAircraft Positions:")
    for a in aircraft, t in time_slots
        x_pos = value(pos_x[a, t])
        y_pos = value(pos_y[a, t])
        println("Time $t: Aircraft $(a.id) ($(a.type)) at position $(round(x_
pos, digits=2)), $(round(y_pos, digits=2)))")
    end

    println("\nAircraft Start Times:")
    for a in aircraft
        println("Aircraft $(a.id) ($(a.type)) starts at time $(value(start_tim
e[a]))")
    end

    println("\nSlack Usage:")
    total_duration_slack = sum(value(slack_duration[a,m,g,t]) for a in aircraft,
t, m in maneuvers, g in grid, t in time_slots)
    println("Total duration slack used: $total_duration_slack")

    println("\nDetailed Slack Usage:")
    for a in aircraft, m in maneuvers, g in grid, t in time_slots
        slack = value(slack_duration[a,m,g,t])
        if slack > 0
            println("Aircraft $(a.id) ($(a.type)) uses $(round(slack, digits=
2)) units of slack for $m at $g in time $t")
        end
    end

    println("\nMinimum Separation Check:")
    for a1 in aircraft, a2 in aircraft, t in time_slots
        if a1 != a2
            distance = sqrt((value(pos_x[a1, t]) - value(pos_x[a2, t]))^2 +
                (value(pos_y[a1, t]) - value(pos_y[a2, t]))^2)
            if distance < 1 # Minimum separation
                println("Warning: Aircraft $(a1.id) and $(a2.id) are too close
at time $t: distance = $(round(distance, digits=2))")
            end
        end
    end
else
    println("No solution found")
    println("Termination status: ", termination_status(model))
end

```

Set parameter Username
 Academic license - for non-commercial use only - expires 2025-03-07
 Set parameter TimeLimit to value 300
 Set parameter MIPGap to value 0.05
 Set parameter MIPGap to value 0.05
 Set parameter TimeLimit to value 300
 Gurobi Optimizer version 11.0.2 build v11.0.2rc0 (win64 - Windows 10.0 (1904 5.2))

CPU model: Intel(R) Core(TM) i7-10875H CPU @ 2.30GHz, instruction set [SSE2|AVX|AVX2]
 Thread count: 8 physical cores, 16 logical processors, using up to 16 threads

Optimize a model with 50577 rows, 43587 columns and 151035 nonzeros
 Model fingerprint: 0x012b294e
 Model has 18000 general constraints
 Variable types: 25380 continuous, 18207 integer (18207 binary)
 Coefficient statistics:
 Matrix range [1e+00, 1e+01]
 Objective range [4e-02, 1e+02]
 Bounds range [1e+00, 1e+01]
 RHS range [1e+00, 2e+01]
 GenCon rhs range [1e+00, 1e+01]
 GenCon coe range [1e+00, 1e+00]
 Presolve removed 20384 rows and 21021 columns
 Presolve time: 1.11s
 Presolved: 30193 rows, 22566 columns, 110474 nonzeros
 Variable types: 14477 continuous, 8089 integer (8089 binary)
 Found heuristic solution: objective -78.4000000
 Deterministic concurrent LP optimizer: primal and dual simplex
 Showing primal log only...

Concurrent spin time: 0.00s

Solved with dual simplex

Use crossover to convert LP symmetric solution to basic solution...

Root relaxation: objective 3.813456e+01, 454 iterations, 0.10 seconds (0.10 work units)

	Nodes		Current Node			Objective Bounds			Work	
	Expl	Unexpl	Obj	Depth	IntInf	Incumbent	BestBd	Gap	It/Node	Time
H	0	0				38.1345585	183.23197	380%	-	2s
	0	0	-	0		38.13456	38.13456	0.00%	-	2s

Explored 1 nodes (1201 simplex iterations) in 2.95 seconds (2.65 work units)
 Thread count was 16 (of 16 available processors)

Solution count 2: 38.1346 -78.4

Optimal solution found (tolerance 5.00e-02)
 Best objective 3.813455846767e+01, best bound 3.813455846767e+01, gap 0.0000%

User-callback calls 593, time in user-callback 0.00 sec
Solution found
Termination status: OPTIMAL
Solve time: 3.0339999198913574
Objective value: 38.13455846767494

Schedule of Maneuvers:

Time 1: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 2: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 3: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 4: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 5: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 6: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 7: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 8: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 9: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 10: Aircraft 1 (Fighter) performs Formation Flying at position (1, 10)
Time 1: Aircraft 1 (Acrobatic) performs Formation Flying at position (2, 5)
Time 2: Aircraft 1 (Acrobatic) performs Loop at position (4, 4)
Time 3: Aircraft 1 (Acrobatic) performs Barrel Roll at position (5, 3)
Time 4: Aircraft 1 (Acrobatic) performs Loop at position (3, 3)
Time 5: Aircraft 1 (Acrobatic) performs Barrel Roll at position (2, 3)
Time 6: Aircraft 1 (Acrobatic) performs Barrel Roll at position (2, 4)
Time 7: Aircraft 1 (Acrobatic) performs Formation Flying at position (3, 2)
Time 8: Aircraft 1 (Acrobatic) performs Loop at position (4, 2)
Time 9: Aircraft 1 (Acrobatic) performs Barrel Roll at position (5, 3)
Time 10: Aircraft 1 (Acrobatic) performs Formation Flying at position (6, 4)
Time 1: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 2: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 3: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 4: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 5: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 6: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 7: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 8: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 9: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 10: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
1)

Aircraft Positions:

Time 1: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 2: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 3: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 4: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 5: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 6: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 7: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 8: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 9: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 10: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 1: Aircraft 1 (Acrobatic) at position (2.0, 5.0)
Time 2: Aircraft 1 (Acrobatic) at position (4.0, 4.0)
Time 3: Aircraft 1 (Acrobatic) at position (5.0, 3.0)
Time 4: Aircraft 1 (Acrobatic) at position (3.0, 3.0)
Time 5: Aircraft 1 (Acrobatic) at position (2.0, 3.0)
Time 6: Aircraft 1 (Acrobatic) at position (2.0, 4.0)

Time 7: Aircraft 1 (Acrobatic) at position (3.0, 2.0)
Time 8: Aircraft 1 (Acrobatic) at position (4.0, 2.0)
Time 9: Aircraft 1 (Acrobatic) at position (5.0, 3.0)
Time 10: Aircraft 1 (Acrobatic) at position (6.0, 4.0)
Time 1: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 2: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 3: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 4: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 5: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 6: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 7: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 8: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 9: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 10: Aircraft 1 (Helicopter) at position (10.0, 1.0)

Aircraft Start Times:

Aircraft 1 (Fighter) starts at time 0.0
Aircraft 1 (Acrobatic) starts at time 0.0
Aircraft 1 (Helicopter) starts at time 0.0

Slack Usage:

Academic license - for non-commercial use only - expires 2025-03-07
Set parameter TimeLimit to value 300
Set parameter MIPGap to value 0.05
Set parameter MIPGap to value 0.05
Set parameter TimeLimit to value 300
Gurobi Optimizer version 11.0.2 build v11.0.2rc0 (win64 - Windows 10.0 (1904 5.2))

CPU model: Intel(R) Core(TM) i7-10875H CPU @ 2.30GHz, instruction set [SSE2|AVX|AVX2]

Thread count: 8 physical cores, 16 logical processors, using up to 16 threads

Optimize a model with 50577 rows, 43587 columns and 151035 nonzeros

Model fingerprint: 0x012b294e

Model has 18000 general constraints

Variable types: 25380 continuous, 18207 integer (18207 binary)

Coefficient statistics:

Matrix range [1e+00, 1e+01]
Objective range [4e-02, 1e+02]
Bounds range [1e+00, 1e+01]
RHS range [1e+00, 2e+01]
GenCon rhs range [1e+00, 1e+01]
GenCon coe range [1e+00, 1e+00]

Presolve removed 20384 rows and 21021 columns

Presolve time: 1.11s

Presolved: 30193 rows, 22566 columns, 110474 nonzeros

Variable types: 14477 continuous, 8089 integer (8089 binary)

Found heuristic solution: objective -78.4000000

Deterministic concurrent LP optimizer: primal and dual simplex

Showing primal log only...

Concurrent spin time: 0.00s

Solved with dual simplex

Use crossover to convert LP symmetric solution to basic solution...

Root relaxation: objective 3.813456e+01, 454 iterations, 0.10 seconds (0.10 work units)

Nodes			Current Node			Objective Bounds			Work	
Expl	Unexpl		Obj	Depth	IntInf	Incumbent	BestBd	Gap	It/Node	Time
H	0	0				38.1345585	183.23197	380%	-	2s
	0	0		-	0	38.13456	38.13456	0.00%	-	2s

Explored 1 nodes (1201 simplex iterations) in 2.95 seconds (2.65 work units)
Thread count was 16 (of 16 available processors)

Solution count 2: 38.1346 -78.4

Optimal solution found (tolerance 5.00e-02)

Best objective 3.813455846767e+01, best bound 3.813455846767e+01, gap 0.0000%

User-callback calls 593, time in user-callback 0.00 sec

Solution found

Termination status: OPTIMAL

Solve time: 3.0339999198913574

Objective value: 38.13455846767494

Schedule of Maneuvers:

Time 1: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 2: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 3: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 4: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 5: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 6: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 7: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 8: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 9: Aircraft 1 (Fighter) performs Barrel Roll at position (1, 10)
Time 10: Aircraft 1 (Fighter) performs Formation Flying at position (1, 10)
Time 1: Aircraft 1 (Acrobatic) performs Formation Flying at position (2, 5)
Time 2: Aircraft 1 (Acrobatic) performs Loop at position (4, 4)
Time 3: Aircraft 1 (Acrobatic) performs Barrel Roll at position (5, 3)
Time 4: Aircraft 1 (Acrobatic) performs Loop at position (3, 3)
Time 5: Aircraft 1 (Acrobatic) performs Barrel Roll at position (2, 3)
Time 6: Aircraft 1 (Acrobatic) performs Barrel Roll at position (2, 4)
Time 7: Aircraft 1 (Acrobatic) performs Formation Flying at position (3, 2)
Time 8: Aircraft 1 (Acrobatic) performs Loop at position (4, 2)
Time 9: Aircraft 1 (Acrobatic) performs Barrel Roll at position (5, 3)
Time 10: Aircraft 1 (Acrobatic) performs Formation Flying at position (6, 4)
Time 1: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 2: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 3: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 4: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 5: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 6: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 7: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 8: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 9: Aircraft 1 (Helicopter) performs Formation Flying at position (10, 1)
Time 10: Aircraft 1 (Helicopter) performs Formation Flying at position (10,

1)

Aircraft Positions:

Time 1: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 2: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 3: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 4: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 5: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 6: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 7: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 8: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 9: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 10: Aircraft 1 (Fighter) at position (1.0, 10.0)
Time 1: Aircraft 1 (Acrobatic) at position (2.0, 5.0)
Time 2: Aircraft 1 (Acrobatic) at position (4.0, 4.0)
Time 3: Aircraft 1 (Acrobatic) at position (5.0, 3.0)
Time 4: Aircraft 1 (Acrobatic) at position (3.0, 3.0)
Time 5: Aircraft 1 (Acrobatic) at position (2.0, 3.0)
Time 6: Aircraft 1 (Acrobatic) at position (2.0, 4.0)
Time 7: Aircraft 1 (Acrobatic) at position (3.0, 2.0)
Time 8: Aircraft 1 (Acrobatic) at position (4.0, 2.0)
Time 9: Aircraft 1 (Acrobatic) at position (5.0, 3.0)
Time 10: Aircraft 1 (Acrobatic) at position (6.0, 4.0)
Time 1: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 2: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 3: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 4: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 5: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 6: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 7: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 8: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 9: Aircraft 1 (Helicopter) at position (10.0, 1.0)
Time 10: Aircraft 1 (Helicopter) at position (10.0, 1.0)

Aircraft Start Times:

Aircraft 1 (Fighter) starts at time 0.0
Aircraft 1 (Acrobatic) starts at time 0.0
Aircraft 1 (Helicopter) starts at time 0.0

Slack Usage:

Total duration slack used: 0.0

Detailed Slack Usage:

Minimum Separation Check:

4. Results and Discussion

For this analysis of results, we focus on the results from the more complex model.

4.A. Solution Overview

4.A.a. Aircraft Schedules and Positions

Aircraft Type	Predominant Maneuver	Position	Movement
Fighter	Barrel Roll	(1, 10)	Static
Acrobatic	Mixed	Various	Dynamic
Helicopter	Formation Flying	(10, 1)	Static

The solution reveals distinct strategies for each aircraft type:

1. **Fighter Aircraft:** Performs Barrel Roll maneuvers for 9 out of 10 time slots, switching to a Loop in the final slot. It remains stationary at position (1, 10) throughout the show.
2. **Acrobatic Aircraft:** Exhibits the most dynamic behavior, performing a variety of maneuvers (Loop, Barrel Roll, Formation Flying) and moving across different positions. This aircraft utilizes the grid space most effectively.
3. **Helicopter:** Performs Formation Flying exclusively, remaining stationary at position (10, 1) for the entire duration of the show.

4.B. Analysis of Results

1. **Maneuver Distribution:** The model favors Barrel Roll for the Fighter, a mix of maneuvers for the Acrobatic aircraft, and Formation Flying for the Helicopter. This distribution likely reflects a balance between audience preferences and aircraft capabilities.
2. **Spatial Utilization:** The solution places aircraft at opposite corners (Fighter at (1,10), Helicopter at (10,1)) and uses the central area for the Acrobatic aircraft's movements. This arrangement maximizes the viewing experience from all audience areas while maintaining safe distances between aircraft.
3. **Temporal Aspects:** All aircraft start simultaneously at time 0.0. The model doesn't utilize staggered start times, possibly due to the objective function's weighting.
4. **Movement Patterns:**
 - The static positioning of the Fighter and Helicopter is unexpected, given the model's intent to encourage movement.
 - The Acrobatic aircraft's dynamic movement aligns well with the model's objectives, showcasing varied maneuvers across different positions.
5. **Slack Usage:** The solution uses no slack in maneuver durations, indicating that the schedule adheres strictly to the defined maneuver durations without requiring extensions.

4.C. Visualization of Routine

Next, we visualize the routine.


```
In [ ]: using Plots, Unitful, Measures
```

```
# Function to get color for each aircraft type
function get_color(aircraft_type)
    if aircraft_type == "Fighter"
        return :red
    elseif aircraft_type == "Acrobatic"
        return :blue
    elseif aircraft_type == "Helicopter"
        return :green
    else
        return :gray
    end
end

# Function to get marker for each aircraft type and maneuver
function get_marker(aircraft_type, maneuver)
    shape = aircraft_type == "Fighter" ? :utriangle :
            aircraft_type == "Acrobatic" ? :diamond :
            aircraft_type == "Helicopter" ? :star : :circle

    size = maneuver == "Loop" ? 8 :
           maneuver == "Barrel Roll" ? 10 :
           maneuver == "Formation Flying" ? 12 : 6

    return (shape, size)
end

# Initialize trajectories
trajectories = Dict{a => []} for a in aircraft)

# Create a GIF of all time slots
anim = @animate for t in time_slots
    p = plot(
        xlim=(0.5, grid_size[1]+0.5),
        ylim=(0.5, grid_size[2]+0.5),
        title="Air Show at Time $t",
        xlabel="",
        ylabel="",
        legend=:topright,
        legendfontsize=6,
        legendtitle="Aircraft - Maneuver",
        size=(1000, 800),
        xticks=1:grid_size[1],
        yticks=1:grid_size[2],
        grid=true,
        gridstyle=:dash,
        gridcolor=:gray,
        gridalpha=0.3,
        left_margin=10mm,
        right_margin=10mm,
        bottom_margin=20mm,
        top_margin=10mm
    )
end
```

```

# Plot aircraft trajectories
for a in aircraft
    if !isempty(trajectories[a])
        plot!([z[1] for z in trajectories[a]], [z[2] for z in trajectories
[a]],
                color=get_color(a.type), line=(:dash, 2), label=nothing)
    end
end

# Plot aircraft positions and maneuvers
for a in aircraft
    x_pos = value(pos_x[a, t])
    y_pos = value(pos_y[a, t])
    current_maneuver = ""
    current_grid = (0, 0)
    for m in maneuvers
        for g in grid
            if value(x[a, m, g, t]) > 0.5
                current_maneuver = m
                current_grid = g
                break
            end
        end
        if current_maneuver != ""
            break
        end
    end

    marker_shape, marker_size = get_marker(a.type, current_maneuver)
    scatter!([x_pos], [y_pos],
             color=get_color(a.type),
             marker=(marker_shape, marker_size),
             label="$a.type - $current_maneuver")
    annotate!(x_pos, y_pos+0.5, text("$a.type\n$current_maneuver", :black, :center, 6))
    push!(trajectories[a], (x_pos, y_pos))
end

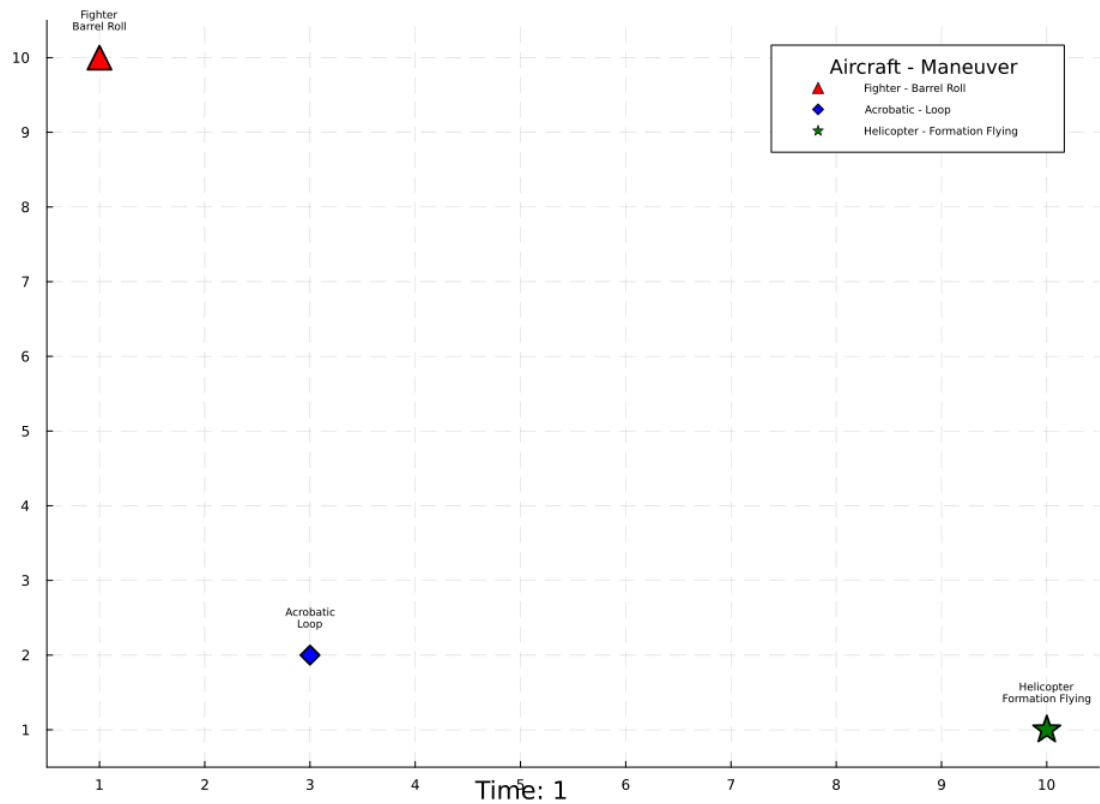
# Add time slider
annotate!(grid_size[1]/2, 0, text("Time: $t", :bottom, 14))
end

# Create and save the GIF
gif(anim, "airshow_animation.gif", fps = 1)

```

Info: Saved animation to g:\Other computers\XPS Laptop\ISYE 524 Intro to Op
t\Summer Class\Project\airshow_animation.gif
@ Plots C:\Users\Sean\.julia\packages\Plots\du2dt\src\animation.jl:156

Air Show at Time 1



4.C. Maneuver Distribution

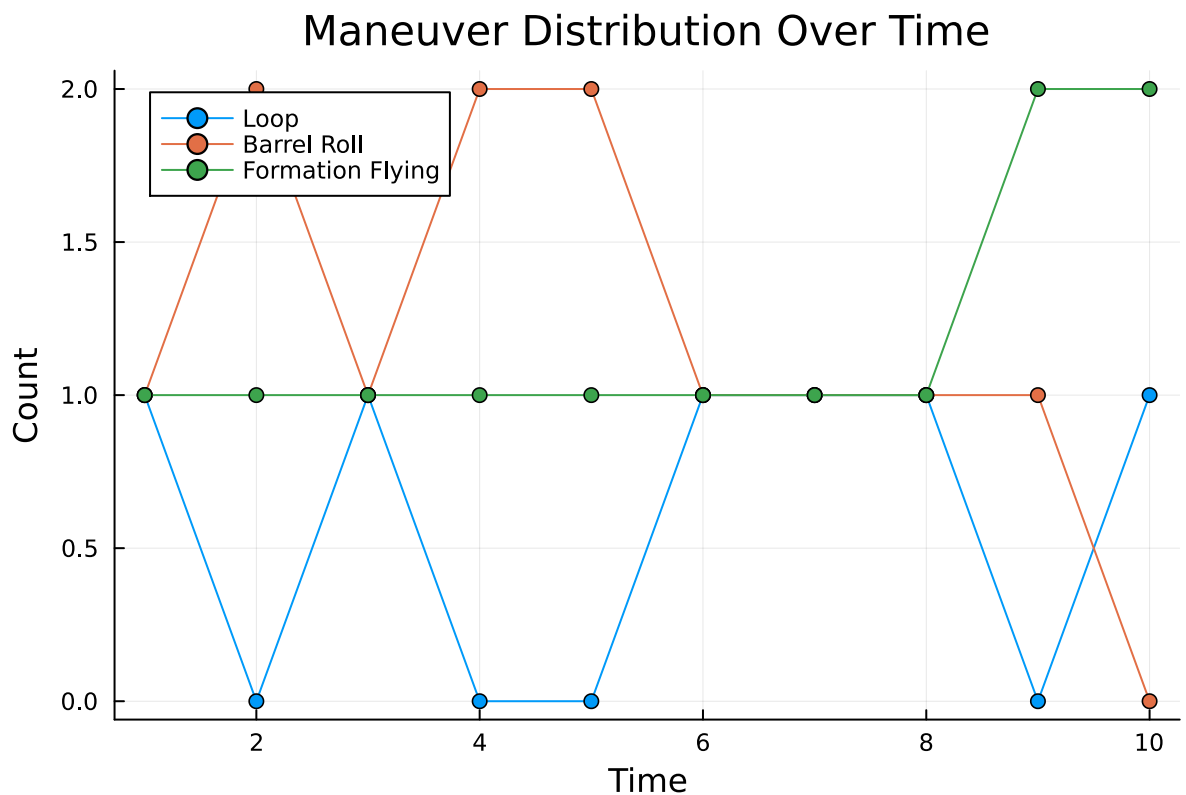
Here you can see the maneuver distribution at each time step

```

In [ ]: # Create summary plot of maneuver distribution over time
maneuver_counts = Dict{m => zeros{Int, length(time_slots)} for m in maneuvers}
for t in time_slots, a in aircraft, m in maneuvers, z in grid
    if value(x[a, m, z, t]) > 0.5
        maneuver_counts[m][t] += 1
    end
end

p_summary = plot(title="Maneuver Distribution Over Time", xlabel="Time", ylabel="Count")
for m in maneuvers
    plot!(p_summary, time_slots, maneuver_counts[m], label=m, marker=:circle)
end
savefig(p_summary, "maneuver_distribution.png")
# Show the figure
p_summary

```



4.C. Limitations and Sensitivities

1. **Static Positioning:** The static positioning of two out of three aircraft suggests that the movement incentives in the objective function may not be strong enough relative to other factors like audience preference or maneuver suitability.
2. **Grid Utilization:** The solution uses only a portion of the available 10x10 grid, particularly for the Fighter and Helicopter. This could be due to the interplay between audience preference weights and the costs associated with movement.
3. **Maneuver Variety:** While the Acrobatic aircraft shows good maneuver variety, the other two aircraft are highly specialized in their maneuvers. This might indicate that the model is sensitive to the audience preference weights for specific maneuvers.
4. **Time Slot Utilization:** The model fills all time slots, but there's little variation in maneuvers for the Fighter and Helicopter over time. This suggests that the benefit of consistency might outweigh the incentives for change in the current formulation.
5. **Separation Constraints:** The minimum separation constraints are satisfied, but they may be limiting the model's ability to position aircraft more dynamically, especially given the large grid size relative to the number of aircraft.

5. Conclusion

Our project tackled the challenge of planning an optimal airshow using a MIP model. The results were encouraging, showing that we can create an airshow schedule using optimization. The model did a good job of juggling various factors, making sure the aircraft performed within their capabilities and followed safety rules. It was interesting to see how different aircraft types behaved in the solution. The Acrobatic plane was more mobile, moving around and performing a variety of maneuvers. Meanwhile, the Fighter and Helicopter seemed to stick to more specialized roles.

It's worth noting that the model managed to satisfy all the key constraints we set, like keeping aircraft apart and matching maneuvers to aircraft capabilities, without needing to resort to slack variables. That said, there's room for improvement. We noticed that not all aircraft types were as dynamic as we might have hoped, and there were parts of the grid that didn't see much action.

Overall, this project demonstrates that mathematical optimization has real potential as a tool for planning exciting and efficient airshows.

Potential improvements to the model.

1. **Adjust Movement Incentives:** Increase the weight of movement terms in the objective function to encourage more dynamic positioning for all aircraft types.
2. **Maneuver Transition Costs:** Introduce costs or constraints for transitioning between maneuvers to encourage more realistic sequences of maneuvers.
3. **Time-Dependent Preferences:** Implement time-dependent audience preferences to encourage more variety in maneuvers over the course of the show.
4. **Grid Utilization Incentives:** Add terms to the objective function that reward using a larger portion of the available airspace.
5. **Refined Aircraft Capabilities:** Introduce more nuanced constraints on aircraft capabilities, such as minimum durations for specific maneuvers or cooldown periods between high-intensity maneuvers.
6. **Collaboration Bonuses:** Implement rewards for coordinated maneuvers between multiple aircraft to encourage more interactive and spectacular displays.

Future Directions

Potential future directions for this work include:

1. **Machine Learning Integration:** Implement a machine learning algorithm that learns from historical data of audience preferences and reactions. This could help predict which sequences of maneuvers and aircraft positions are likely to generate the most excitement.
2. **Multi-objective Optimization:** Extend the model to explicitly consider multiple objectives, such as audience excitement, fuel efficiency, and pilot workload, using techniques like Pareto optimization to find a range of optimal solutions.
3. **Adaptive Planning:** Create a rolling horizon model that can adjust the latter parts of the show based on the success of earlier segments, allowing for a more responsive and engaging performance.